UNIVERSITÀ DI PISA

MASTER DEGREE IN COMPUTER ENGINEERING

ELECTRONICS AND COMMUNICATIONS SYSTEMS

Pseudo Noise Sequence Generator

Student:

Klaudio Çaça

Academic Year 2024/25

TABLE OF CONTEXT

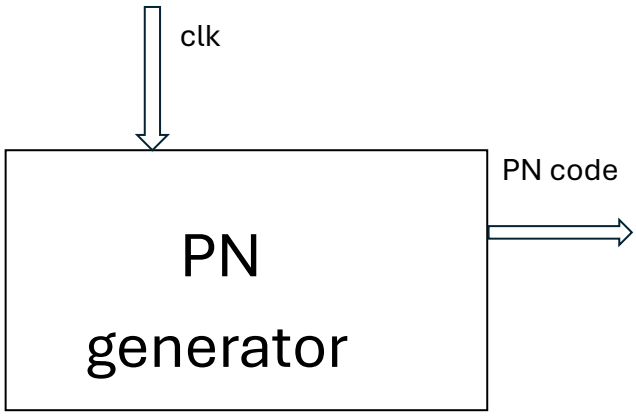
TABLE OF CONTEXT 2
OBJECTIVES 3
KEY CONCEPT AND APPLICATIONS 3
POSSIBLE ARCHITECTURE OF A PNSG 4
OPERATIONS OF THE ARCHITECTURE CHOSEN FOR THE IMPLEMENTATION OF THE PNSG FOR CDMA IS-95 TRANSMISSIONS 7
IMPLEMENTATION OF THE PN GENERATOR 7
MODELSIM VERIFICATION 13
RTL ANALYSIS AND SYNTESIS 17
CONCLUSION 20
REFERENCES 21

OBJECTIVES

The objective of this project is to implement a Pseudo Noise Sequence Generator for CDMA IS-95 transmission using the VHDL language and synthesize it on the Zync FPGA. The generator must be compatible with

1) 15 stage PN generator
2) Characteristic polynomial: $x^{15} + x^{13} + x^9 + x^8 + x^7 + x^5 + 1$

Schematic block:



KEY CONCEPT AND APPLICATIONS

A Pseudo Noise Sequence Generator PNSG (also called Pseudo Random Number Generator PRNG) is an electronic device which can transform a given input called *key* or *seed* into a longer output called *keystream*. Its output must look like a random bitstream, so it must satisfy one or more standard tests for statistical randomness. Even though the PNSG output looks like it hasn't any particular pattern, the pseudorandom noise actually follows a deterministic sequence of outputs. The only thing which is random itself is the *seed*. The generator uses an algorithm to expand this value in a longer sequence of bits. Because of this, not only the bit-stream is deterministically produced, but it also starts replicating itself after a certain number of steps, called <u>period</u> of the pseudo-random sequence. The latter depends on the algorithm used to implement the generator.
Suppose that our *seed* it's composed by n bits, then the maximum period can only reach $2^n - 1$ bits.

PNSG devices represent a category of tools that continue to acquire more and more greater relevance in the development of new technologies; the need to exploit sequences of pseudo-random number is now manifested in multiple applications, including:

- Encryption (key generation)
- Geometric (Monte Carlo method)
- Modelling of financial developments (stock market shares)
- Computer engineering (neural network training, initialization algorithms)
- Telecommunications (broadcast "Spread Spectrum", which we will discuss)
- Electronics (frequency device modelling)
- Entertainment (video games, gambling)

POSSIBLE ARCHITECTURE OF A PNSG

One of the most widely used architectures, especially in the field of cryptography, is the one based on the circuit implementation of a device known as the Linear Feedback Shift Register (LFSR), of which I report a representation (Figure 1).

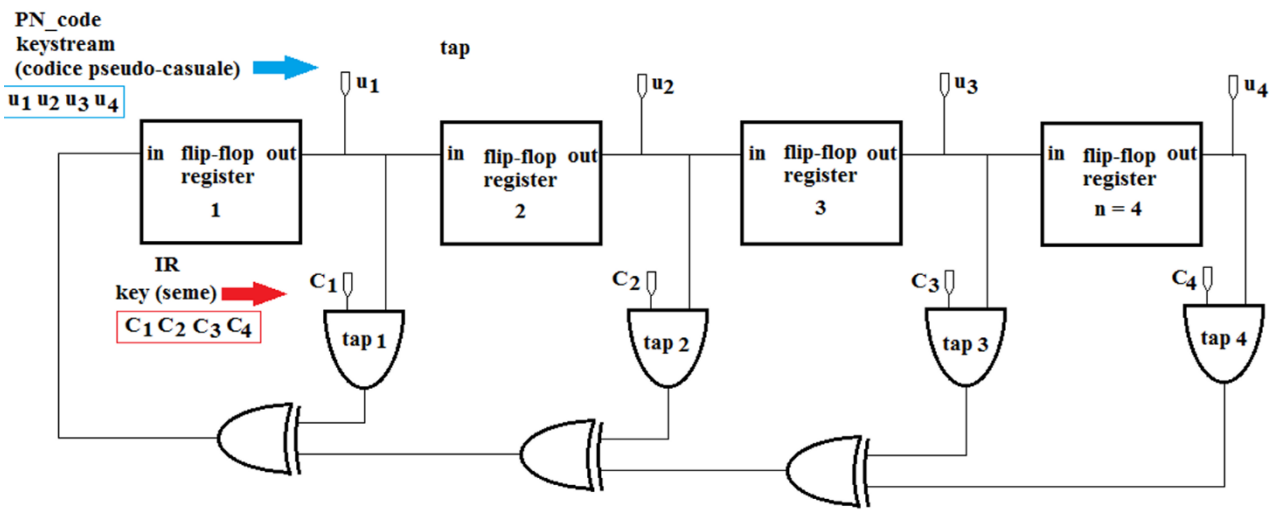


Figure 1: Schematic principle of a Linear Feedback Shift Register (LFSR).

The bits that make up the *seed* $(C_1, \dots, C_n)$ are inserted in parallel to set the initial state of the device; the *key* must be entered before the start of the signal clock, which is common to all registers arranged in cascade. Before the first rising edge of the clock occurs (let's assume we have flip flop positive edge triggered registers), the *reset* (active low) of each serial register must have steadily assumed the logical value "1". Starting from the first rising edge of the clock onwards, i.e. from the instant in which, for the first time, each register updates the value of its output with that of its input, the state of the device evolves over time: the LFSR transits, clock period after clock period, from one state to another, until it returns to the initial value,

after a certain period, which depends on the seed inserted in the initial state loaded via the Ci terminations of the n AND ports.

In the field of mobile telecommunications ("wire-less communications") it is used often a particular version of the LFSR, known as PNSG, which is my choice for the realization of the generator of pseudo-random sequences for transmissions CDMA IS-95. In figure 2 there is an abstract representation of a generic PNSG built using the architecture I have chosen and described, while on figure 3 there is the "schematic view", which is the detailed circuit diagram (described in VHDL language), of a 15-stage PNSG (15 D-flip-flop registers) compactly represented by the characteristic polynomial G(x) = $x^{15} + x^{13} + x^9 + x^8 + x^7 + x^5 + 1$, and on the left the "symbolic view" of the PNSG itself.

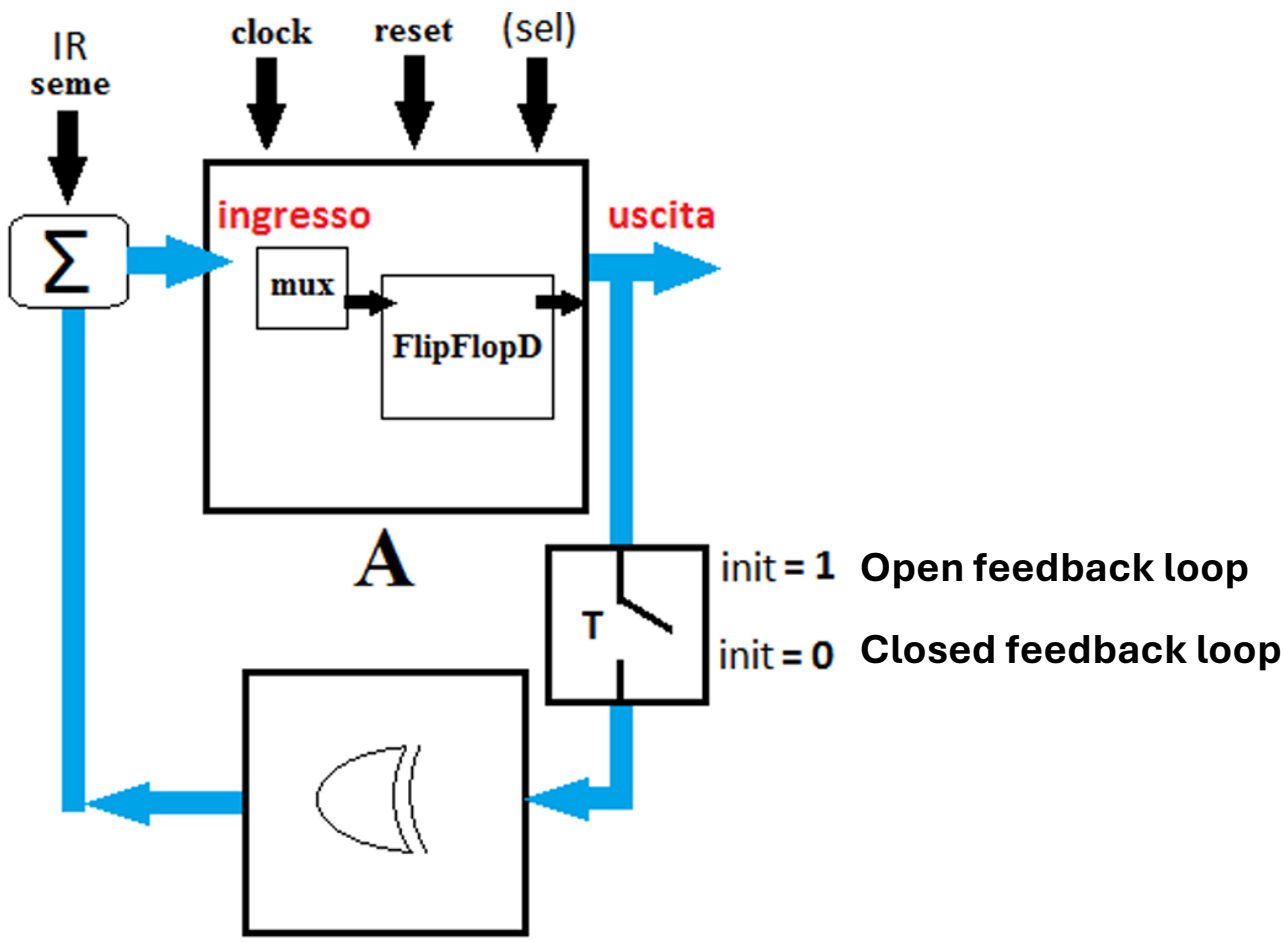


Figure 2: abstract representation of a generic PNSG.

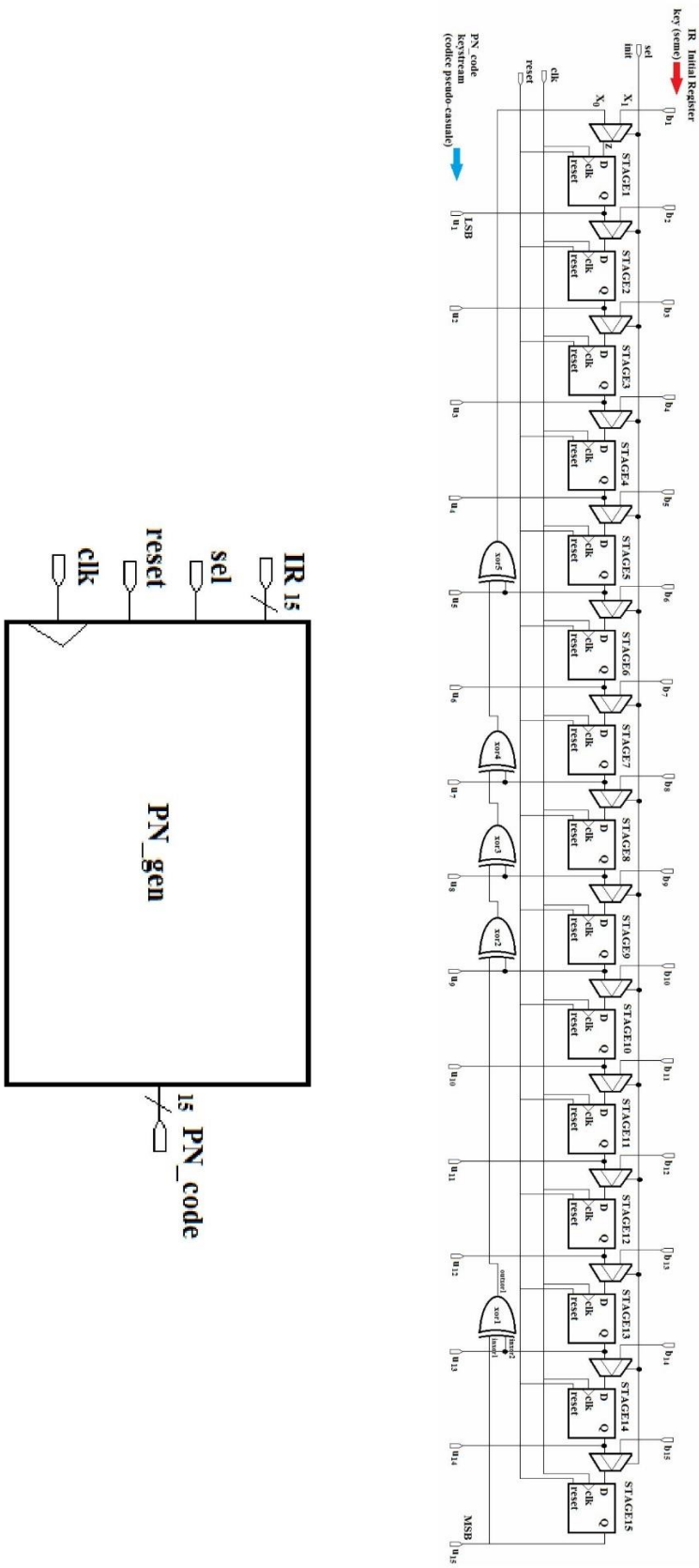


Figure 3: On the left the symbolic view, on the right the schematic view using VHDL language.

OPERATIONS OF THE ARCHITECTURE CHOSEN FOR THE IMPLEMENTATION OF THE PNSG FOR CDMA IS-95 TRANSMISSIONS

It is crucial to make sure that the reset signal, taken by us active low, which is common to the 15 serial registers D flip-flop, has taken on a stable logical value "1" at the instant when, for the first time, the clock signal undergoes a positive edge variation, while the value of "sel" already has a stable assumed value "0", i.e. the feedback loop is closed. If at the instant of the first rising edge of the clock the reset was low all 15 flip-flops D would update their respective outputs Q, each of consist of a single bit, with the logical value "0", therefore the initial state of the PNSG would result in a string of 15 zeros and it would be impossible for the device to evolve, over time, into a different state, so to exit the initial loaded state.

Some instants before the first rise edge of the clock, it is crucial to have inserted the 15 bits *seed* and to have set the control bit *sel* to “0”. When sel is “0” the multiplexer choses the input *x1* which is connected to the input of the *seed*, while if sel is setted to “1” the multiplexer choses *x0* which is the output Q of the previous D flip flop (see again figure 3). So before the first rising edge of the clock the bits *x0* of every multiplexer has got the respective bit of the *seed*. So from now on the *sel* bit must be put to “0” in order to make the system evolve clock after clock based on the logic of the characteristic polynomial chosen G(x).

Architecturally, the structure of the PNSG is very similar to that of the LFSR: the difference in fact consists of the methods of inserting the *seed*, carried out precisely by the multiplexers and not through the AND gates, and in the fact that the Ci bits of the LFSR are rigidly made of static circuit connections, in order to implement the algebraic form of G(x). In the LFSR if we don’t want the monomial x^{14} of G(x) we must set the bit C_{14} to “0”, so that the AND port will be “0”. In PNSG architecture we simply avoid connecting C_{14} to the nearest port XOR. In the LSFR we use 15 AND ports, in the PNSG we only use 5 XOR ports in this specific case of G(x).

As I said in the introduction the maximum period is $2^n - 1$ bits. G(x) is a polynomial of factor 15, so it means that my maximum period is $2^{15} - 1$ which is 32767. With some research and experiments it has been noted that choosing this *seed 000000000000001* the maximum period is reached.

IMPLEMENTATION OF THE PN GENERATOR

In this section, the VHDL implementation of the previously described generator is showed. The first basic elements needed are the flip flop, which will store the value of the i-th stage of the generator, and the binary multiplexer, that will filter the proper input to its correspondent flip flop based on the value of the signal *sel*, as discussed in the previous section.

D_flip_flop.vhd: VHDL file of a simple flip flop positive edge triggered.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

Entity D_flip_flop is
    port(
        d       : in std_logic;
        clk     : in std_logic;
        reset   : in std_logic;
        q       : out std_logic
    );
end entity;

architecture beh of D_flip_flop is

    begin
        P_dff : process (clk, reset)
        begin
            if (reset = '0') then
                q <= '0';
            elsif (rising_edge(clk)) then
                q <= d;
            end if;
        end process P_dff;

end architecture;
```

Mux.vhd: VHDL File of a simple binary multiplexer.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

Entity Mux is
    port(
        x0  : in std_logic;
        x1  : in std_logic;
        sel : in std_logic;
        z   : out std_logic
    );
end entity;
```

```
architecture beh of Mux is

    begin
        P_mux : process (x0, x1, sel)
        begin
            if (sel = '0') then
                z <= x0;
            elsif (sel ='1') then
                z <= x1;
            end if;
        end process P_mux;

end architecture;
```

PNG.vhd: VHDL File of the Pseudo Noise Sequence Generator.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

Entity PNG is
    generic (
        N: positive := 15
    );
    port (
        clk     : in std_logic;
        init    : in std_logic;
        IR      : in std_logic_vector (1 to N);
        reset   : in std_logic;
        PN_code : out std_logic_vector (1 to N)
    );
end entity;

architecture structural of PNG is
    Component D_flip_flop
    port (
        d       : in std_logic;
        clk     : in std_logic;
        reset   : in std_logic;
        q       : out std_logic
    );
    end component;
```

```
Component Mux
    port(
        x0      : in std_logic;
        x1      : in std_logic;
        sel     : in std_logic;
        z       : out std_logic
    );
    end component;

    signal qmux : std_logic_vector (1 to N);
    signal inXor : std_logic_vector (1 to 6);
    signal muxd : std_logic_vector (1 to N);
    signal outXor : std_logic_vector (1 to 5);

begin
    PNG_structure: for i in 1 to N generate

        FIRST: if i = 1 generate
            MX1: Mux port map(
                    x0 => outXor(5),
                    x1 => IR(i),
                    sel => init,
                    z => muxd(i)
                );
            FF1 : D_flip_flop port map(
                    d => muxd(i),
                    clk => clk,
                    reset => reset,
                    q => qmux(i)
                );
            PN_code(i) <= qmux(i);
        end generate FIRST;

        INTERNAL: if i > 1 and i < N generate
            MXi: Mux port map(
                    x0 => qmux(i-1),
                    x1 => IR(i),
                    sel => init,
                    z => muxd(i)
                );
            FFi : D_flip_flop port map(
                    d => muxd(i),
                    clk => clk,
                    reset => reset,
                    q => qmux(i)
                );
            PN_code(i) <= qmux(i);
        end generate INTERNAL;
```

```
        LAST: if i = N generate
            MXN: Mux port map(
                    x0 => qmux(i-1),
                    x1 => IR(i),
                    sel => init,
                    z => muxd(i)
                );
            FFN : D_flip_flop port map(
                    d => muxd(i),
                    clk => clk,
                    reset => reset,
                    q => inXor(1)
                );
            PN_code(i) <= inXor(1);
        end generate LAST;

    end generate PNG_structure;

    OutPortXor1:
        inXor(2) <= qmux(13);
        outXor(1) <= inXor(1) xor inXor(2);
        -- inXor(1) <= q of last FFD;
        -- OutPortXor1 = q15 XOR q13;

    OutPortXor2:
        inXor(3) <= qmux(9);
        outXor(2) <= outXor(1) xor inXor(3);
        -- OutPortXor2 = OutPortXor1 XOR q9;

    OutPortXor3:
        inXor(4) <= qmux(8);
        outXor(3) <= outXor(2) xor inXor(4);
        -- OutPortXor3 = OutPortXor2 XOR q8;

    OutPortXor4:
        inXor(5) <= qmux(7);
        outXor(4) <= outXor(3) xor inXor(5);
        -- OutPortXor4 = OutPortXor3 XOR q7;

    OutPortXor5:
        inXor(6) <= qmux(5);
        outXor(5) <= outXor(4) xor inXor(6);
        -- OutPortXor5 = OutPortXor4 XOR q5;

end architecture structural;
```

In my PNG code I have simply initialized the component *D_flip_flop* and *Mux* that I have already described in the above VHDL files. Then I declared the signals (wires) that I need for proper connections between the Muxs and the Flip Flops. I generate my fifteen stages by connecting their multiplexers and flip flops properly with each wire. Finally, I simply take the outputs of the stages as the output of my PNG and do the XOR operation on the specific output Q as requested by the characteristic polynomial G(x).

PNG_tb.vhd: VHDL File of the testbench to correctly verify the architecture.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity PNG_tb is
end entity;

architecture beh of PNG_tb is
    component PNG
        generic (
            N: positive := 15
        );
        port (
            clk     : in std_logic;
            init    : in std_logic;
            IR      : in std_logic_vector (1 to N);
            reset   : in std_logic;
            PN_code : out std_logic_vector (1 to N)
        );
    end component;

    constant clk_period : time      := 10 ns;
    constant TestLen    : positive  := 66000;
    constant N          : positive  := 15;

    SIGNAL clk_ext : std_logic := '0';
    SIGNAL init_ext : std_logic := '1';--init to 1 (sel = 1 to all of the 15 mux)
    signal IR_ext : std_logic_vector (1 to N):= "000000000000001";
    signal reset_ext : std_logic := '0'; --flip flop deactivated, i 15 q to zero
    signal PNcode_ext : std_logic_vector (1 to N);
    signal clk_cycle : integer; -- clock counter
    signal Testing : Boolean := True;

    begin
        clk_ext <= not clk_ext after clk_period/2 when testing else '0';
```

```
c_DUT: PNG
        generic map(
            N => 15
        )
        port map(
            clk => clk_ext,
            init => init_ext,
            IR => IR_ext,
            reset => reset_ext,
            PN_code => PNcode_ext
        );

    Test_Proc : PROCESS (clk_ext)

        variable count : INTEGER := 0;

        BEGIN
            clk_cycle <= (count+1)/2;

            case count is
                when 3 => reset_ext <= '1';

                when 10 => init_ext <= '0';

                when (TestLen-1) => Testing <= False;

                when others => NULL;
            end case;

        count := count + 1;

    end Process Test_Proc;

end architecture;
```

When count is 3, it means at the second clock cycle, the reset is set to “1”, so from the next rising edge of the clock onwards the system can change its output clock after clock.

When count is 10, it means we are right before the six rising edge of the clock, and the init_ext (sel) is reset to “0”, which means that from now on the multiplexer will choose the input x0, which is the output of the previous stage.

MODELSIM VERIFICATION

In this section I will use the ModelSim tool to properly see the functionality of my code and to verify that it works as expected. Below are reported the results.

In Figure 4 we can see the initialization of our wires: PN_code is properly initialized to “0”, the *reset* is active low, IR which is the *seed* is set to 000000000000001 and finally init which is *sel* is initiated to 0 so that the multiplexer allows only the bits of the *seed* to the flip flops.

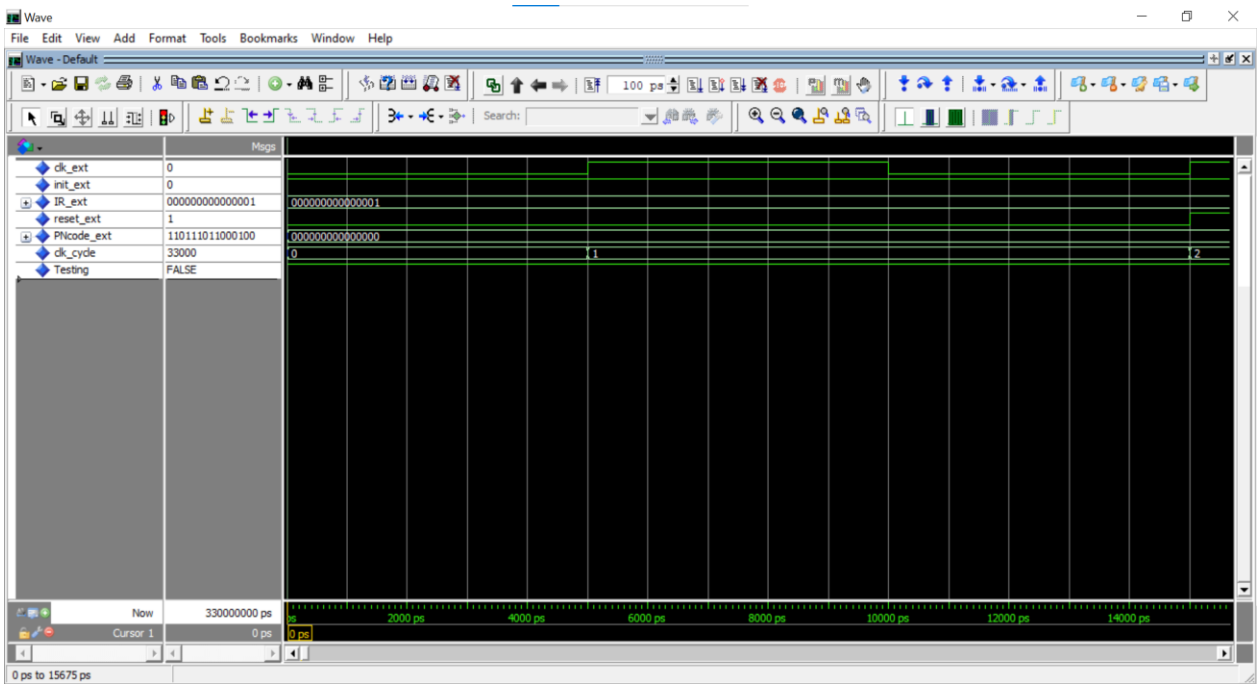


Figure 4: Initialization of wires.

In the Figure 5 is shown that the *reset* is set to “1”, so at the next rising edge of the clock we can see that PN_code has properly initialized with the *seed* (IR_ext).

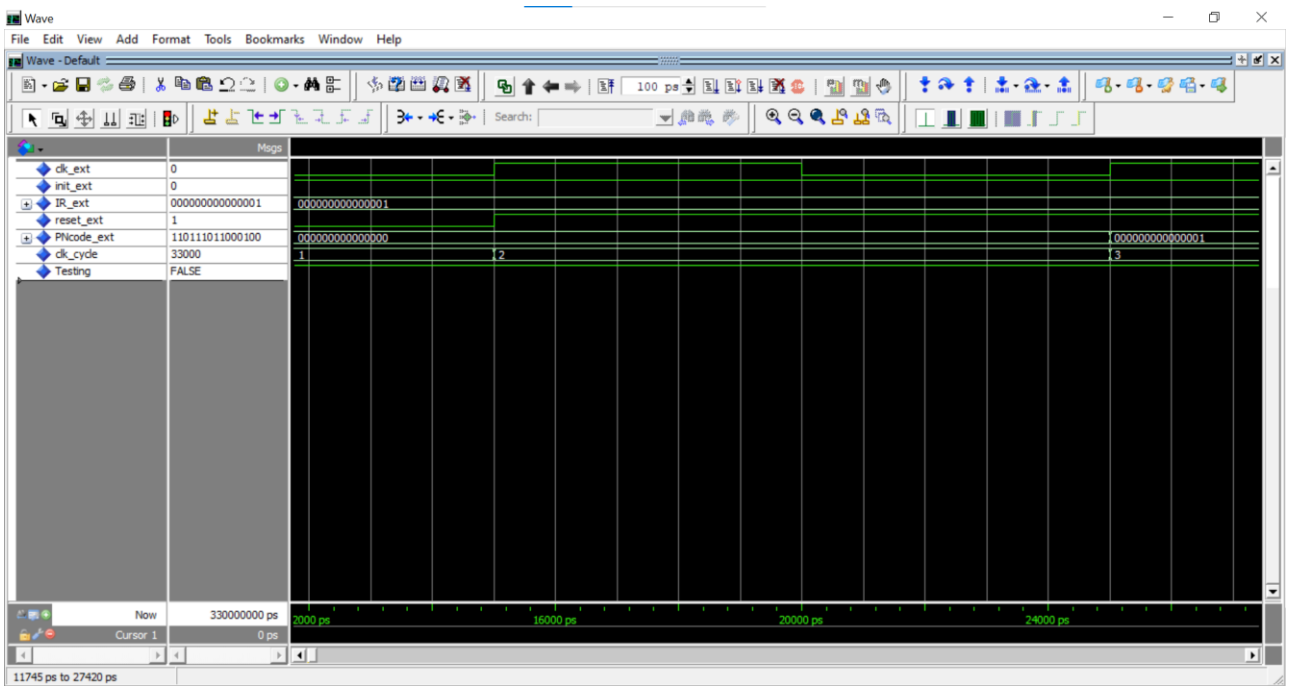


Figure 5: The reset wires is set to “1”.

In Figure 6 is shown that the *init* wire (which is *sel*), is resetted to “0”, so from the next edge of the clock the PN_code can evolve following the criteria of the polynomial G(x). See that at *clk_cycle* 6 the PN_code evolves from 00000000000001 to 100000000000000.

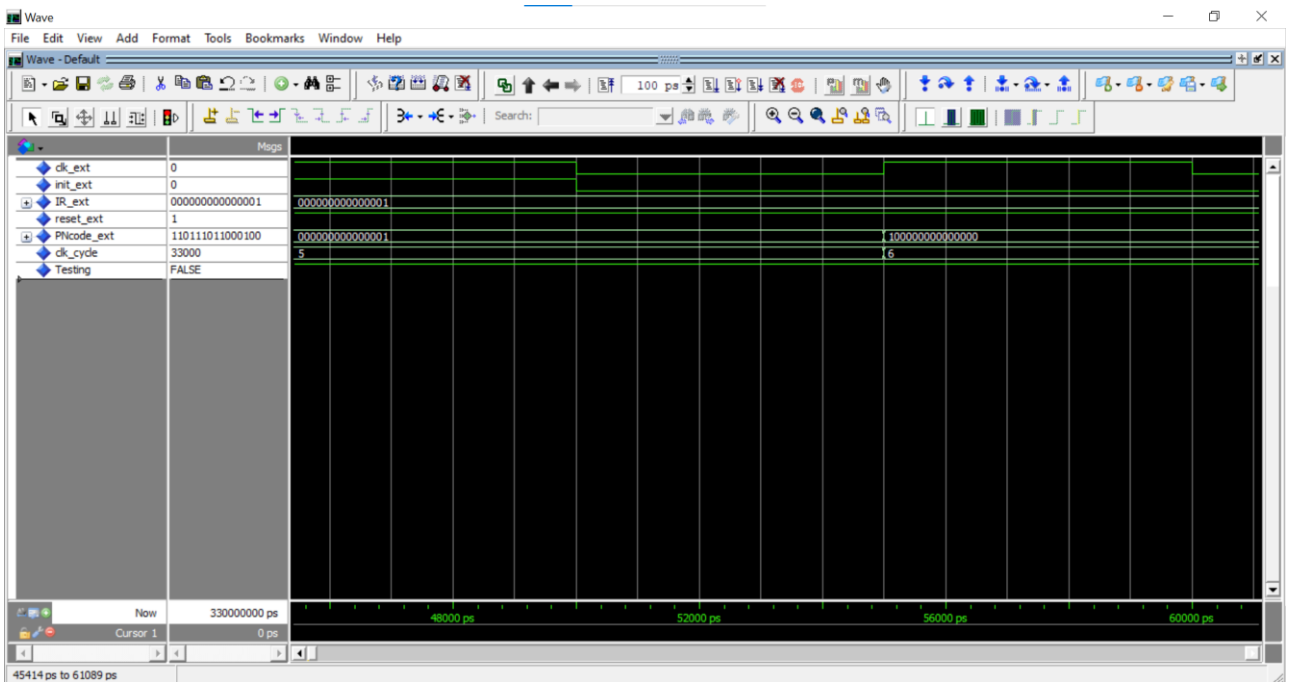


Figure 6: The init wire (sel) is reset to “0”.

In figure 7 we can see that the system evolves like a shift register of the least significant beat until the 11th clock cycle, exactly 5 clock cycle after the first change in output of PN_code. This simply is the first effect of the feedback loop which is the XOR5 operation, as expected.

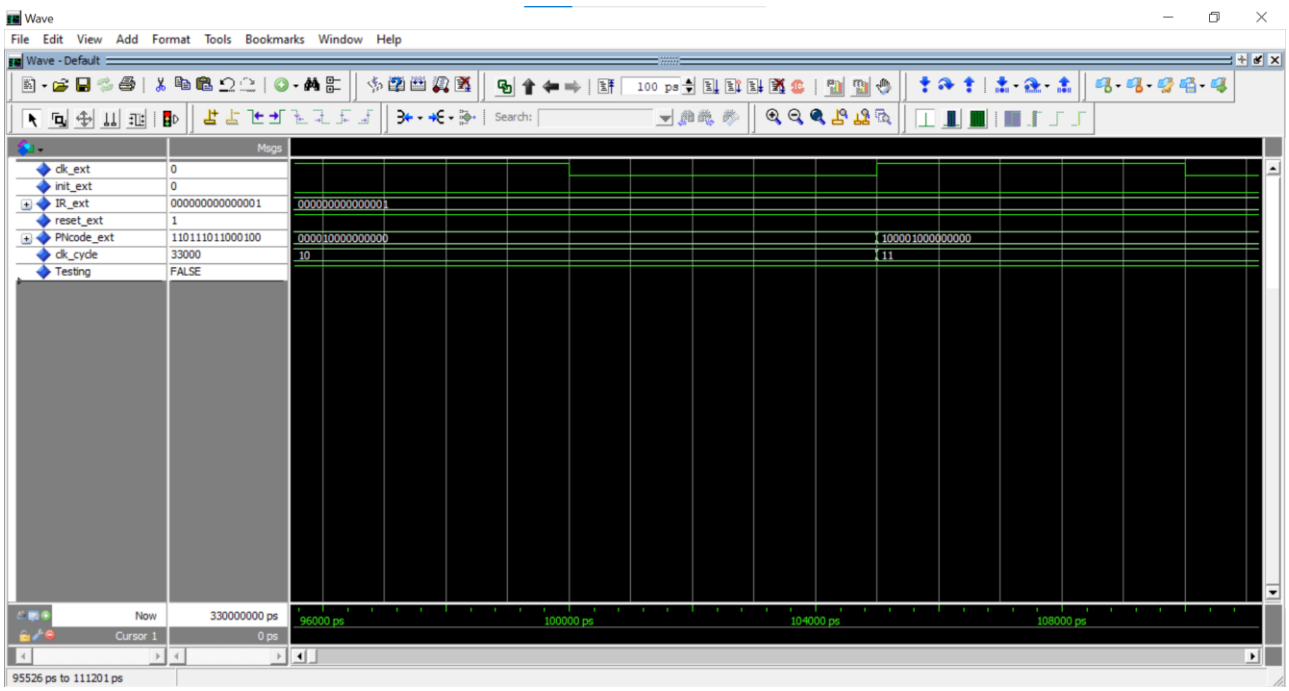


Figure 7: First feedback loop at 11th clock cycle.

Finally in Figure 8 the periodicity of the PNSG is shown. We deactivated the *sel* bit at the 5th clock cycle, so we expect to encounter the first value (the value of the seed) 00000000000001 after exactly 32767 clock cycle, which is 32767 + 5 = 32772.

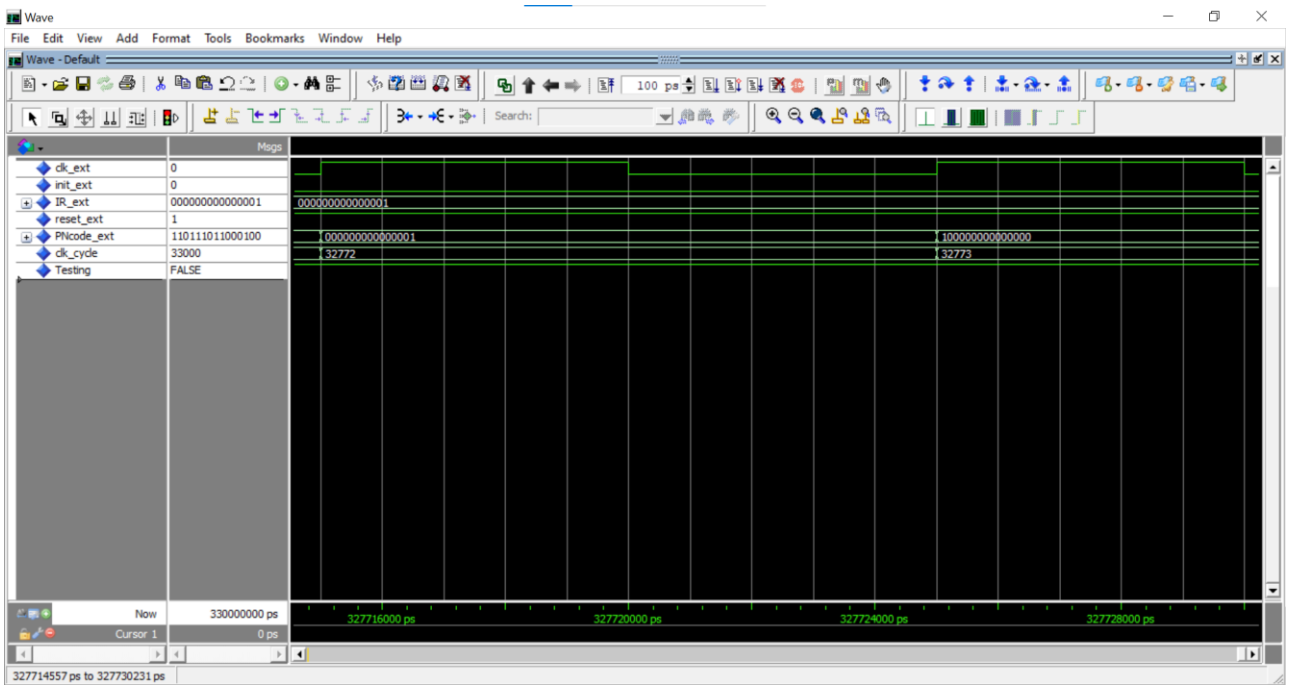


Figure 8: Periodicity of 32767 clock cycle.

RTL ANALYSIS AND SYNTESIS

To synthesize the designed PN generator on the Zynq FPGAs family, Vivado tool was used. An RTL (Register-Transfer-Level) analysis is automatically done by Vivado.

In the next Figure (Figure 9), Vivado has properly connected the input wires, which are the *clock*, *reset*, *init* and the *seed* to all the *Muxs* and *Flip flops*. We can see the XOR ports and if we zoom we can see that the XOR operations involves the outputs q15, q13, q9, q8, q7, q5 as expected. In Figure 10 is reported the connection of the output PN_code.

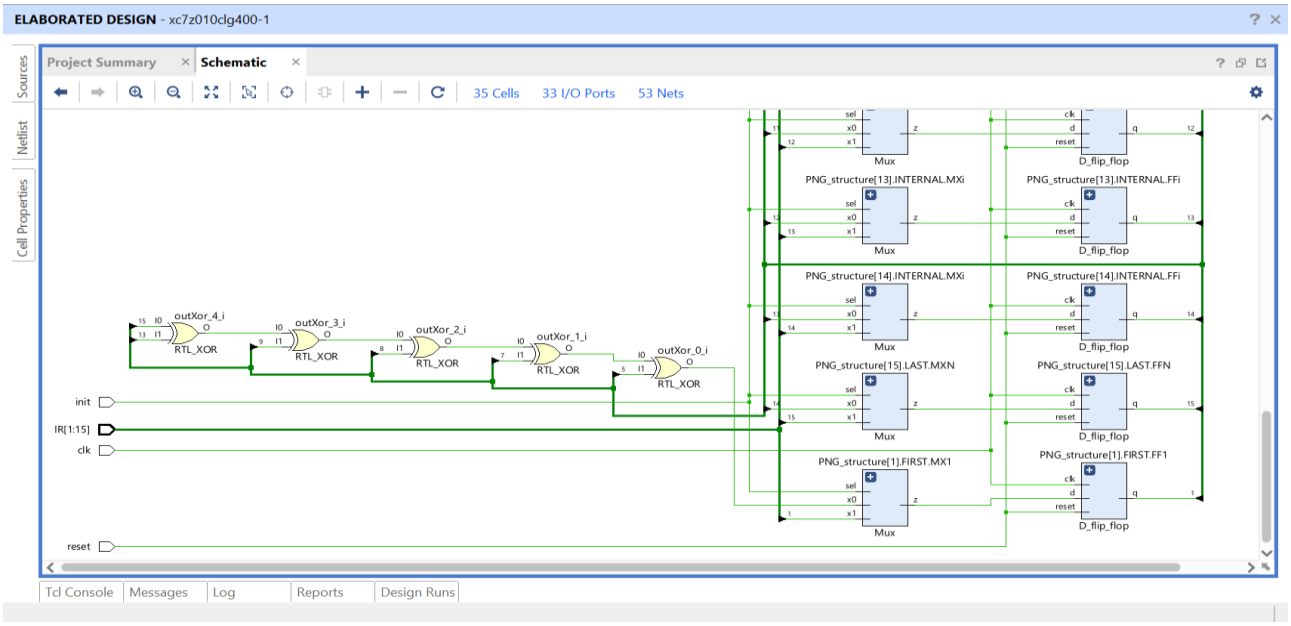


Figure 9: Elaborated design of PNSG.

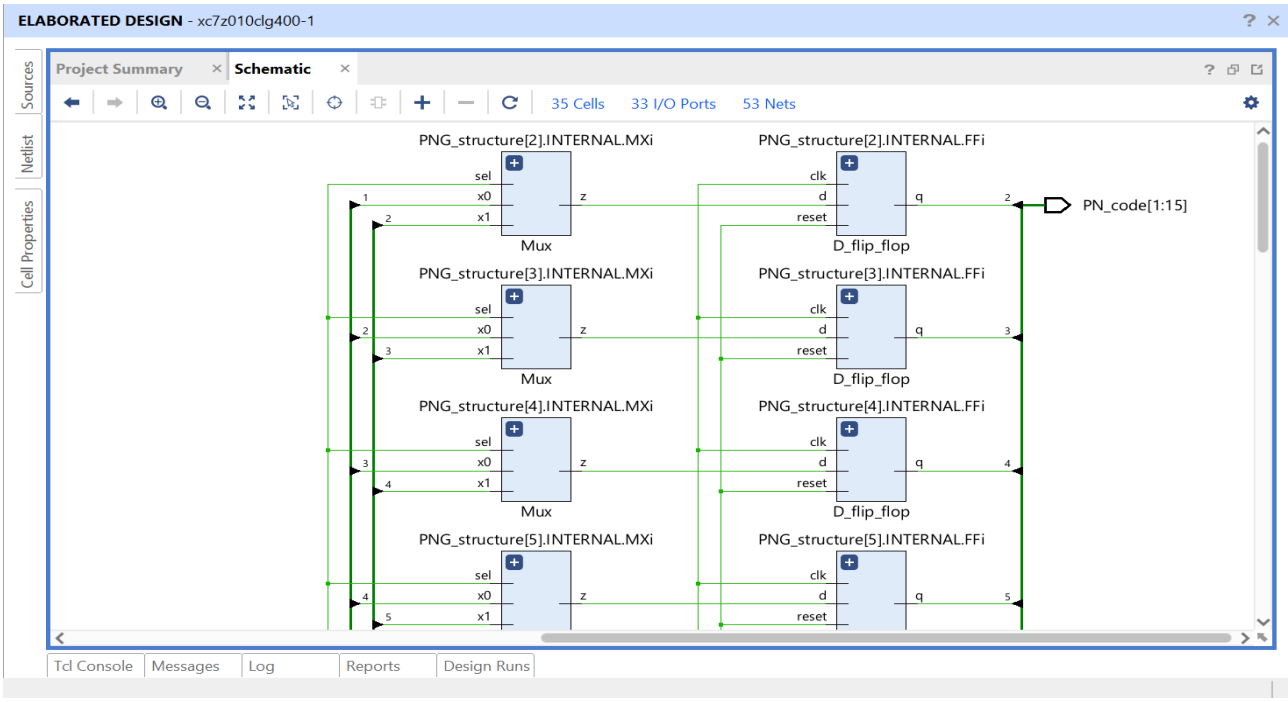


Figure 10: output of PN_code.

After a first RTL analysis, the synthesis of the board has been run: the selected target frequency of the clock has been set up to $f^* = 100\ MHz$. After the synthesis, the resulting WNS (Worst Negative Slack), computed by Vivado considering the critical path, is ***7,638 ns***. In the figures below we can see the WNS and the slack of each path.

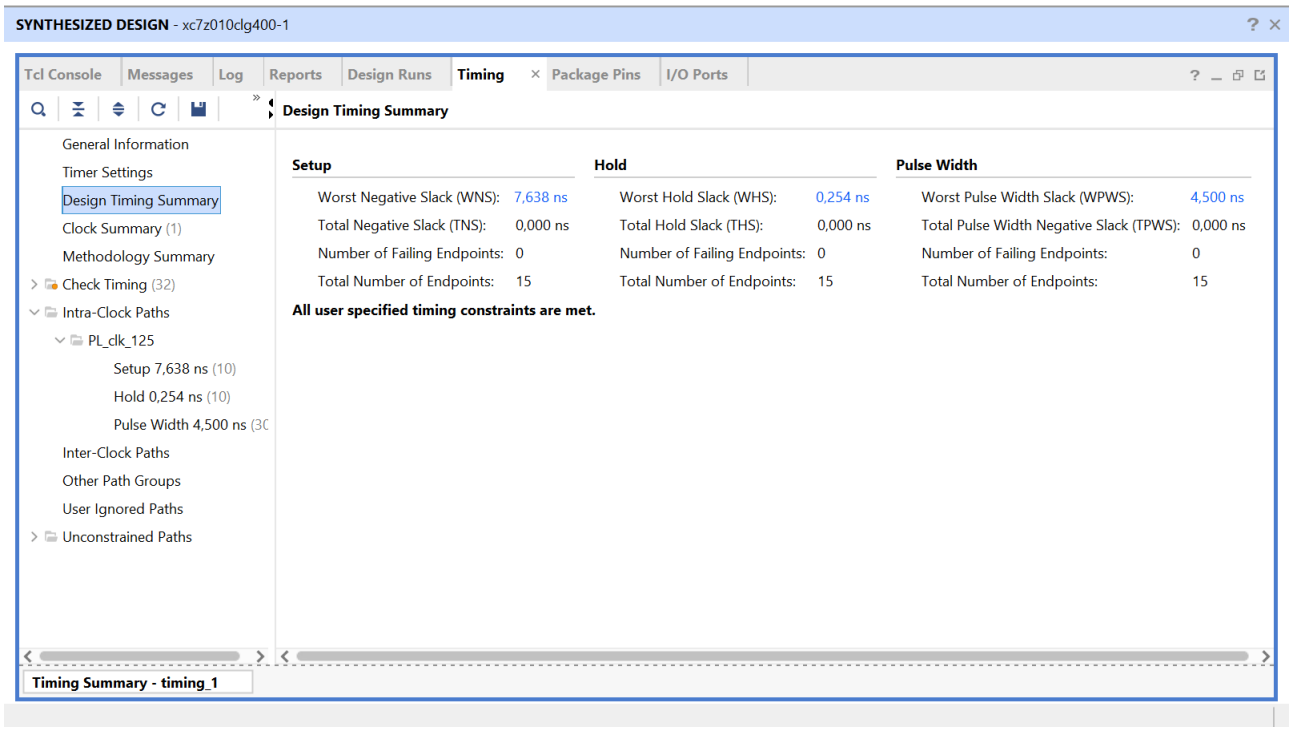


Figure 11: Worst Negative Slack.

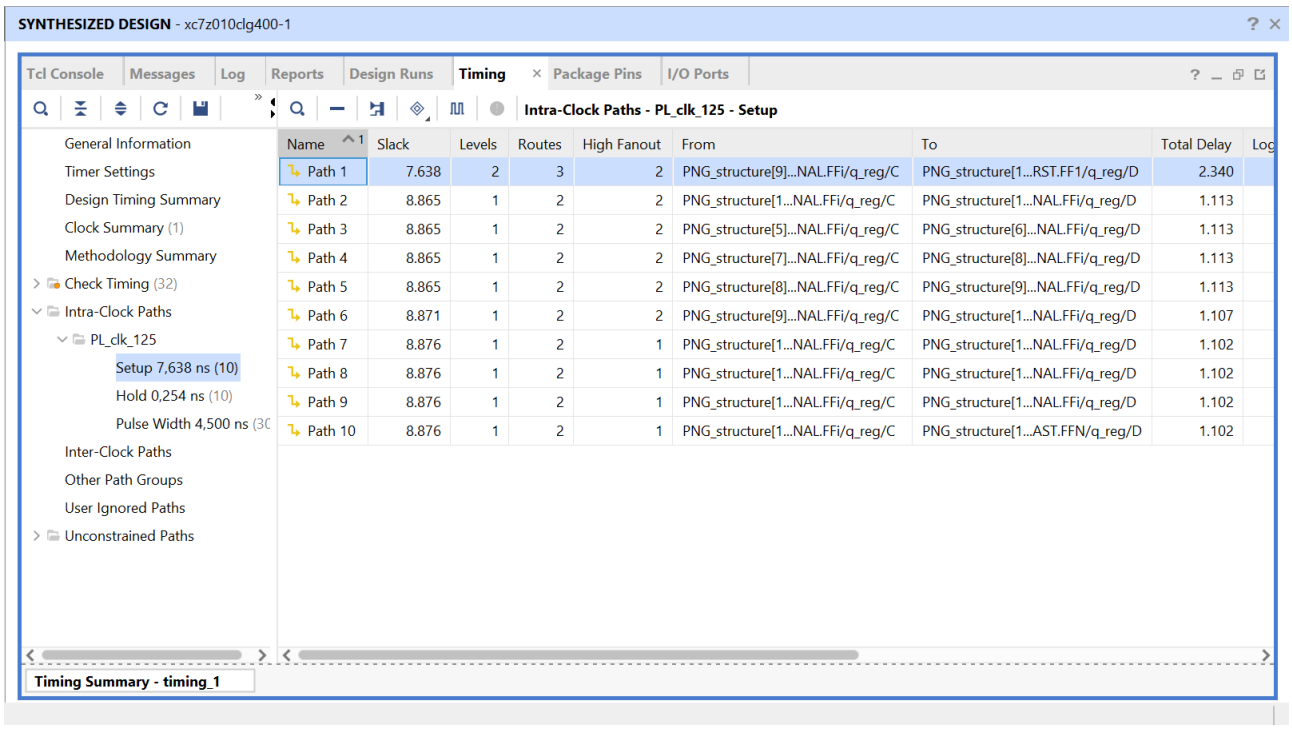
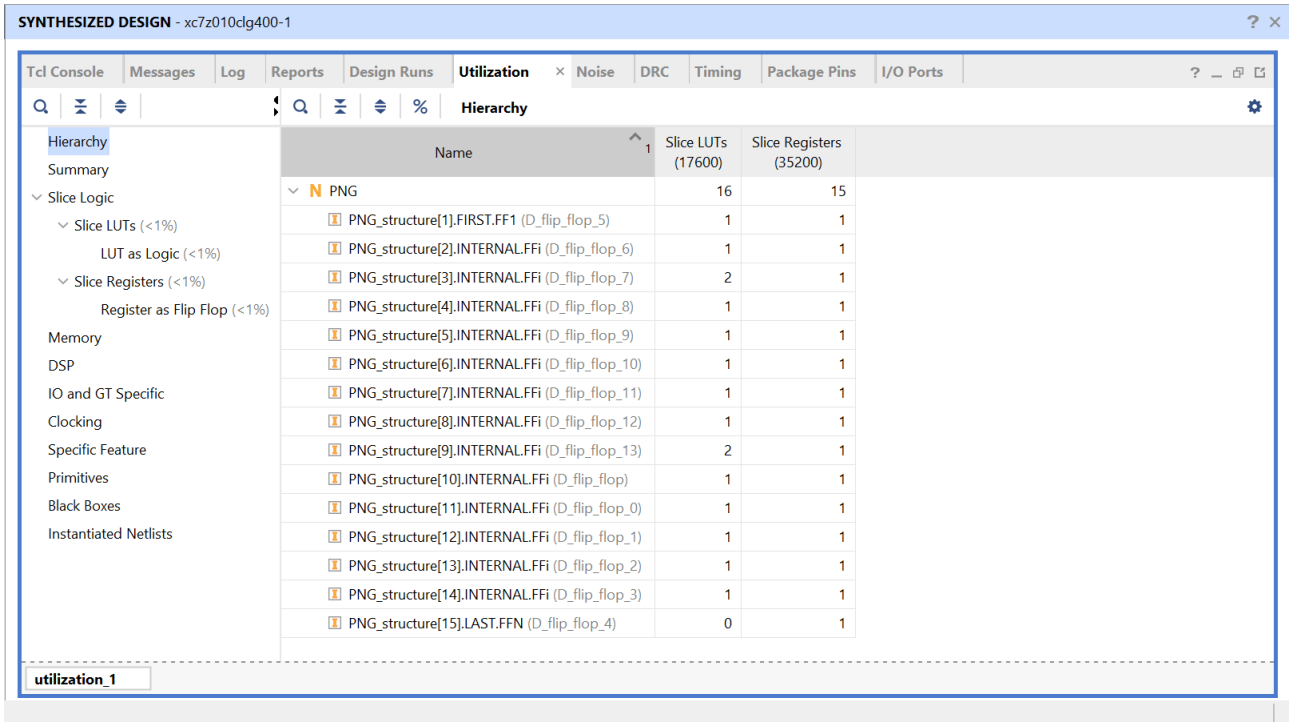


Figure 12: Slacks of each path.

The critical path is enlightened with the first path, which is the path from register 9 to 1. Once the WNS is obtained, the maximum operating frequency can be computed as:

$$f_{clk}^{max} = \frac{1}{T_{clk}^{*} - WNS} \cong 423{,}37\ MHz$$

The Vivado tool also analysed the used resources and the power consumption, that are reported below in Figure 13:



Name	Slice LUTs (17600)	Slice Registers (35200)
PNG	16	15
PNG_structure[1].FIRST.FF1 (D_flip_flop_5)	1	1
PNG_structure[2].INTERNAL.FFi (D_flip_flop_6)	1	1
PNG_structure[3].INTERNAL.FFi (D_flip_flop_7)	2	1
PNG_structure[4].INTERNAL.FFi (D_flip_flop_8)	1	1
PNG_structure[5].INTERNAL.FFi (D_flip_flop_9)	1	1
PNG_structure[6].INTERNAL.FFi (D_flip_flop_10)	1	1
PNG_structure[7].INTERNAL.FFi (D_flip_flop_11)	1	1
PNG_structure[8].INTERNAL.FFi (D_flip_flop_12)	1	1
PNG_structure[9].INTERNAL.FFi (D_flip_flop_13)	2	1
PNG_structure[10].INTERNAL.FFi (D_flip_flop)	1	1
PNG_structure[11].INTERNAL.FFi (D_flip_flop_0)	1	1
PNG_structure[12].INTERNAL.FFi (D_flip_flop_1)	1	1
PNG_structure[13].INTERNAL.FFi (D_flip_flop_2)	1	1
PNG_structure[14].INTERNAL.FFi (D_flip_flop_3)	1	1
PNG_structure[15].LAST.FFN (D_flip_flop_4)	0	1

Figure 13: Used resources of the PNSG synthetised by Vivado.

A total of 16 Slice LUTs and 15 Slice Registers have been used.

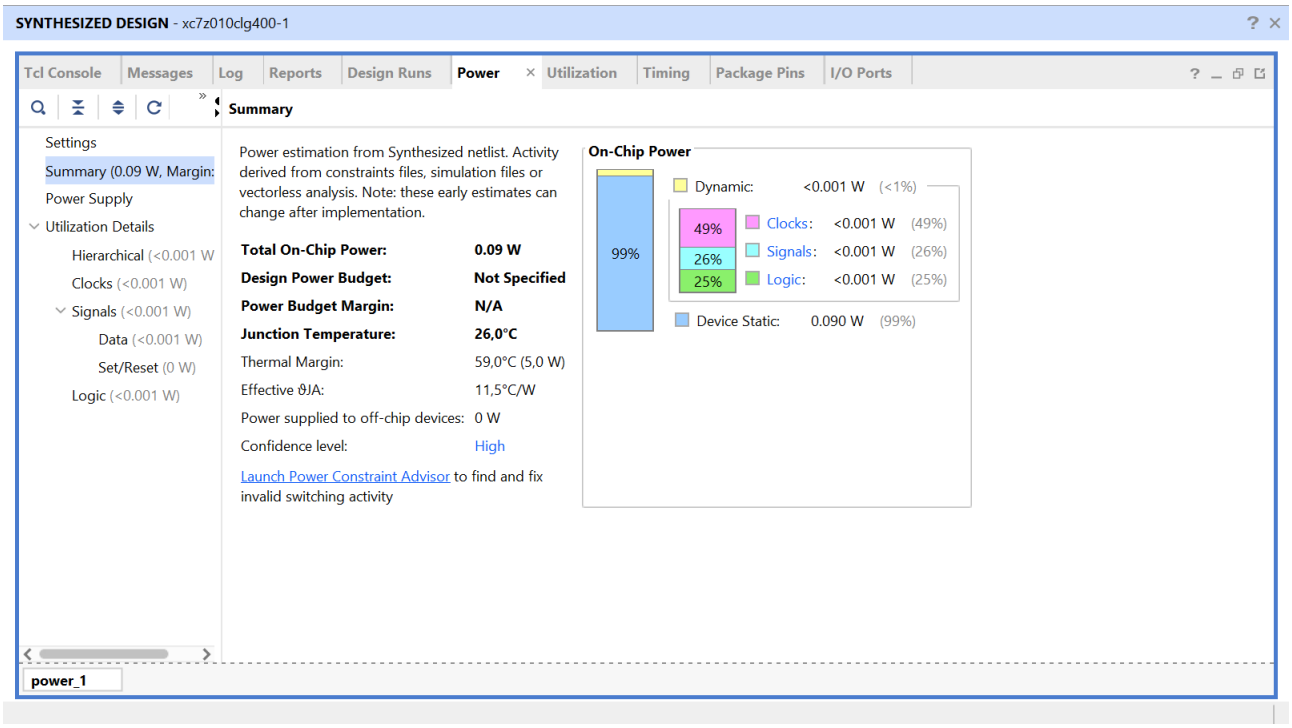


Figure 14: Power Consumption.

CONCLUSION

This project shows how a Pseudo Random Noise Generator works and the characteristic it must have to work properly; it shows some possible implementations starting from the Linear Feedback Shift Register (LFSR) to my final architectural choice which is the Pseudo Noise Sequence Generator PNSG, specifically the one for CDMA IS-95 transmissions.

I showed the basic functioning of this PNSG and provided the VHDL code implementation along with the testbench to verify the correctness of my code.

Finally, I used ModelSim to properly visualize the output of my implementation and to assert the correctness of the PNSG characteristics. Also Vivado tool was used to make a RTL analysis, generate all the possible paths and select the Worst Negative Slack (WNS), and from this to compute the maximum clock frequency on which the system can operate. Utilization and power consumptions were also displayed.

In conclusion, this project shows a possible implementation of a Pseudo Random Noise Generator with respect to the telecommunication environment, but this exact implementation could be used also in very different fields, as discussed in the initial section of this report. This makes the implementation very versatile and exploitable in different contexts.

There exist many versions of Pseudo Noise Sequence Generators that are similar to our study case, their implementation wouldn't be so different from the one presented.

REFERENCES

[1] MathWorks

https://it.mathworks.com/help/comm/ref/pnsequencegenerator.html

[2] Wikipedia

https://en.wikipedia.org/wiki/Linear-feedback_shift_register
https://en.wikipedia.org/wiki/CdmaOne

[3] Matematicamente

https://www.matematicamente.it/staticfiles/approfondimenti/Molinari-sequenze_pseudo-casuali.pdf